

ECE 537:

Applied Data Analytics

Exercise #1:

Generating Gaussian Data Set and Data Classes

Gaussian distributions and Gaussian data are commonly used in the teaching of data analysis methods as they provide (or admit) relatively simple mathematical solutions that remain mathematically tractable.

Real world data sets tend to have far more complex distributions as well as multi-modal behaviors. But, developing a solid understanding of data analysis within Gaussian context provides a solid starting point to developing the knowledge and skills required to analyze more complex data.

For this in-class exercise you need to do the following work while:

- Using Matlab's Live Script feature.
- Working in groups of two.

Tasks:

1. Develop a matlab function that creates K samples of D -dimensional Gaussian data with arbitrary specified mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$.
2. Fully test your developed function to ensure that it indeed produces D -dimensional Gaussian data with the *correct* mean vector and covariance matrix.
3. Discuss why, when $K < \infty$, the Matlab's computed covariance matrix for your Gaussian generated data will *not* exactly match the covariance you prescribed.
4. Use your created function to create 1000 data samples for each of the three classes A, B, and C where each is described by a 3-dimensional Gaussian distributions with their sufficient statistics given as:

$$\boldsymbol{\mu}_A = \begin{bmatrix} 0 \\ 4 \\ 7 \end{bmatrix}, \boldsymbol{\mu}_B = \begin{bmatrix} 10 \\ 5 \\ 5 \end{bmatrix}, \boldsymbol{\mu}_C = \begin{bmatrix} -2 \\ -5 \\ -5 \end{bmatrix}$$

and

$$\boldsymbol{\Sigma}_A = \begin{bmatrix} 6.5000 & 2.1651 & 1.2500 \\ 2.1651 & 5.1250 & 2.3816 \\ 1.2500 & 2.3816 & 2.3750 \end{bmatrix}, \boldsymbol{\Sigma}_B = \begin{bmatrix} 3.0184 & 0.4331 & -1.4481 \\ 0.4331 & 0.9436 & -0.6456 \\ -1.4481 & -0.6456 & 1.2880 \end{bmatrix},$$

$$\boldsymbol{\Sigma}_C = \begin{bmatrix} 6.8928 & -1.3750 & 3.1945 \\ -1.3750 & 3.3572 & 0.6945 \\ 3.1945 & 0.6945 & 3.7500 \end{bmatrix}$$

The data you produce must *exactly* match the above means and covariances, i.e., when you run Matlab's `cov()` function you should exactly get $\boldsymbol{\Sigma}_A$, $\boldsymbol{\Sigma}_B$, and $\boldsymbol{\Sigma}_C$ and not covariances that are simply close to these stated covariances.

5. You need to create your own function to generate the required Gaussian data using Matlab's `randn()` function which produces $N(0, 1)$ Gaussian data samples. Once you have created your function you can then compare your results to the results produced via Matlab's `mvnrand()`

function. Are the results similar? Why are they not exactly the same? Which function exactly produced data sets with the required Σ_A , Σ_B and Σ_C covariances? Why is it important to be able to generate data with *exactly* the covariance that you require (and not just covariances that are “close”) ?

6. Use matlab’s ellipsoid() function to create contour intervals for Classes A, B, and C at one, two, and three standard deviations from μ_A , μ_B , and μ_C and also on the same plot the principal axes of each generated Gaussian contour ellipses using matlab’s line() function.
7. Use the provided vol3D.m function to create a regular set of uniformly spaced grid points within the 3D volume of the Class A, B, and C Gaussian distributions. This function will return a vector of 3D data points within the volume you have specified and you can use the matlab plot3d() function to plot these data points.
8. Apply the class discriminant function to this set of vol3D() data points and assign each point to its respective class based on Bayes classification.
9. Draw the classification boundary by identifying the points in the 3D volume for which the difference between discriminate functions is less than a defined threshold, i.e., $|g_1(x) - g_2(x)| < \epsilon$.

As you are dealing with real data and sample data points within the 3D volume the theoretical Bayes boundary, which occurs at $g_1(x) - g_2(x) = 0$, is not directly accessible via the data, i.e., the tested 3D volume data points are highly unlikely to fall exactly on the class decision boundaries.

Instead, the above approach allows you to identify a sufficiently close neighborhood of the ideal Bayes boundary by the use of the ϵ term. In general, and $\epsilon \in [10^{-4}, 10^{-2}]$ should work reasonably well depending on the sampling density you used to produce your vol3D data samples.

10. Your produced 3-class boundary should look identical to the boundary image provided on the ECE 537 Brightspace page. The only difference that should arise should be due solely to the sampling density and ϵ threshold used. Any significant difference is the result of errors within your code.

Notes:

- You are expected to build your own solution to generating random Gaussian data with prescribed means and covariance. Marks will not be given for using Matlab's build-in functions for this. Instead you should based you solution around the whitening transform and inverse whitening transform process.
- The produced boundary is relatively complex even though it is just the Bayes boundary of three simple Gaussian. The boundary though is far too complex to solve for analytically.

Hence, the approach you are taking is to sample the 3D volume in which the Gaussian distributed data lies and then use the discriminant function approach to determine for each data point which class it then belongs to. It is through finding the boundary points of this classification process that you are able to produce the plot of the decision boundary and its shape.

Do not try to analytically solve for this boundary as that approaches only works for highly trivial cases.

The Whitening Transform

The whitening transform is used to convert data with arbitrary covariance Σ into data with $\Sigma' = \mathbf{I}$ covariance. For Gaussian data this converts the ellipsoid contour intervals into circular intervals, thereby making the data linearly uncorrelated via the transform,

$$D' = T_W[D - \mu] + \mu$$

As $T_W[\cdot]$ is a scaling and rotational transform it must be performed on data that has first been shifted to have its centre (mean) at the origin, the transform is then applied, and the resulting data is then shifted back to its original centre (mean). If you do not first shift the data such that it is zero mean before applying whitening then scaling and rotation about a non-zero mean will introduce an additional translation. The resulting data will therefore be wrong.

Given a set of data $D = \{\mathbf{x}_n | n = 1, \dots, N\}$ with sample mean $\hat{\mu}$ and sample covariance $\hat{\Sigma}$. The eigendecomposition of the covariance matrix is then given by $\hat{\Sigma} = \Phi \Lambda \Phi^T$ and Φ is the matrix of $\hat{\Sigma}$'s eigenvectors and Λ is the diagonal matrix of $\hat{\Sigma}$'s eigenvalues.

The whitening transform is then defined as,

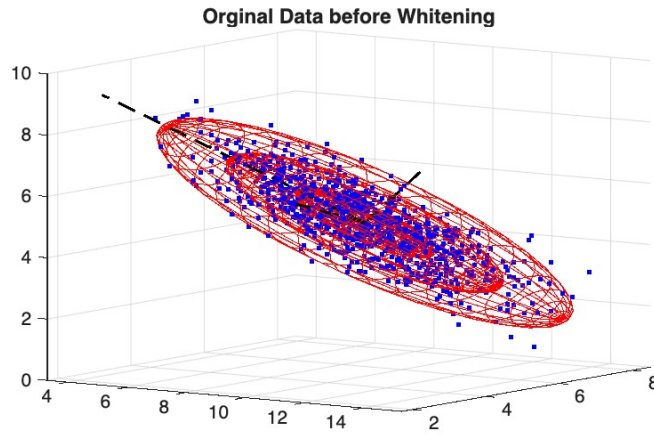
$$T_W = \Lambda^{\frac{1}{2}} \Phi^T$$

or

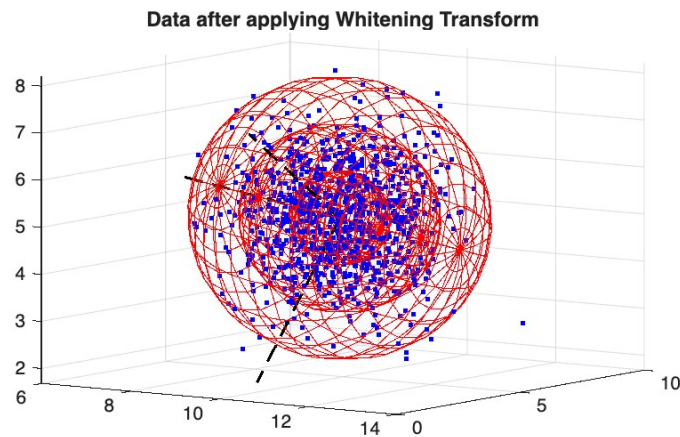
$$T_W = \Phi \Lambda^{\frac{1}{2}} \Phi^T$$

where the second form is used primarily in image processing tasks to ensure minimal pixel movement occurs between the original data D and the transformed data $D' = T_W[D - \mu]$.

Note: The above transform is also a reversible process, i.e., it has a $T_W^{-1}[\cdot]$ inverse. Hence, if one had zero mean data with an identity covariance matrix, i.e., Gaussian data that had circular standard deviation contour intervals, then one could use the $T_W^{-1}[\cdot]$ inverse transform of the $T_W[\cdot]$ whitening transform to be able to convert that $N(0, 1)$ data *exactly* into any arbitrary Gaussian distribution that you wanted to produce, i.e., when you run Matlab's `cov()` function on the resulting data it should *exactly* produce the covariance you desired the data to have.



(a) Example of a Gaussian distribution with arbitrary mean and covariance.



(b) Example of a Gaussian distribution with arbitrary mean and identity covariance.

Figure 1: An example of an arbitrary Gaussian distribution pre- and post-application of the whitening transform.

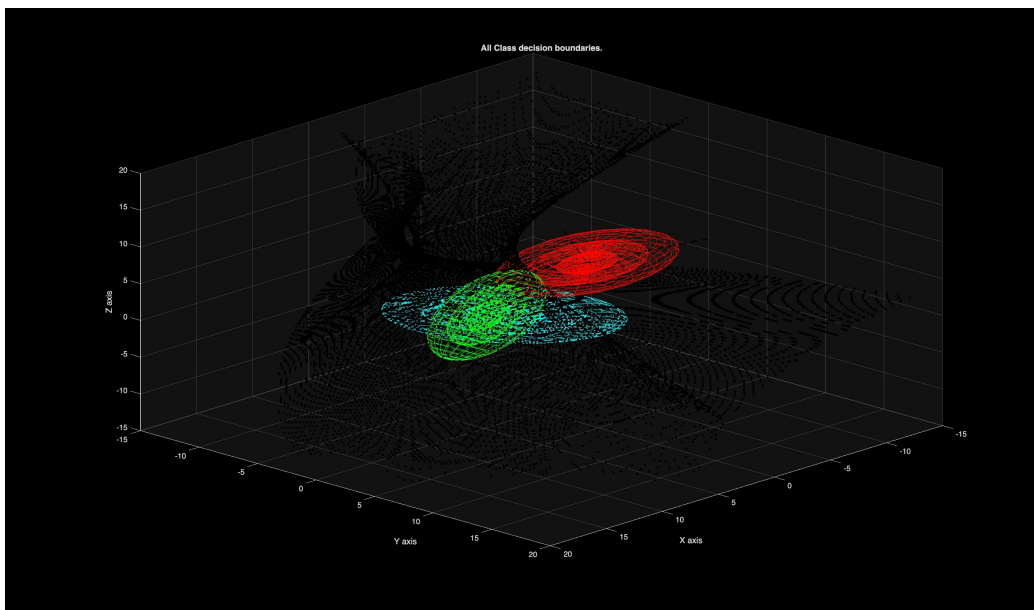


Figure 2: Bayes decision boundary for Classes A, B, and C assumed equal a priori probabilities.